



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارش کار آزمایش دهم

عنوان:

پیاده‌سازی و توسعه یک پردازنده ساده

نام درس

آزمایشگاه طراحی سامانه‌های رقمی

تابستان ۱۴۰۵

استاد

دکتر اجلالی

دستیار استاد

علی اثنی عشری

گروه ۲

علی سلطانی - ۴۰۳۱۰۶۰۹۲

یگانه کربلایی - ۴۰۳۱۰۶۵۲۷

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	هدف آزمایش	۲
۲	مشخصات معماری و قرارداد پشته	۲
۱.۲	ساختار کلی پردازنده	۲
۲.۲	مجموعه دستورها	۳
۳.۲	نمودار بلوکی پردازنده توسعه یافته	۳
۳	توسعه مسیر داده برای سه دستور جدید	۴
۱.۳	دستور SWAP	۴
۲.۳	دستور SHL	۴
۳.۳	دستور CMP	۴
۴.۳	جدول ریز عملیات دستورهای جدید	۵
۴	توسعه واحد کنترل	۵
۵	نگاشت حافظه و ورودی و خروجی	۶
۶	ساختار فایل های اصلی و کدهای قابل سنتز	۷
۱.۶	هسته پردازنده: stack_cpu.v	۷
۲.۶	حافظه و درگاه ها: memory_io.v	۱۶
۳.۶	سامانه پردازنده: processor_system.v	۲۰
۴.۶	رمزگشای نمایشگر هفت قسمتی: seven_segment_decoder.v	۲۱
۵.۶	پیمانه سطح بالا: stack_processor_top.v	۲۲
۷	برنامه زبان ماشین	۲۴
۱.۷	خانه های موقت حافظه	۲۴
۲.۷	منطق محاسبه تابع	۲۴
۳.۷	تبدیل خروجی به دو رقم مبنای شانزده	۲۴
۴.۷	حداکثر عمق پشته	۲۵
۵.۷	نمودار روند برنامه	۲۶
۶.۷	جدول کامل کد ماشین	۲۶
۸	سنتز و مشاهده ساختار در Quartus	۲۹
۱.۸	روند انجام کار در Quartus	۲۹
۲.۸	نمای سطح بالای سامانه	۳۰
۳.۸	نمای تفصیلی پردازنده	۳۱
۴.۸	نتیجه کامپایل	۳۱
۹	طراحی و روش کار فایل های آزمون	۳۲
۱.۹	آزمون مستقل دستورهای جدید در stack_cpu_tb.v	۳۲
۱.۱.۹	آزمون SWAP	۳۲
۲.۱.۹	آزمون های SHL	۳۲

۳۲	 ۳.۱.۹ آزمون‌های CMP
۳۳	 ۲.۹ آزمون برنامه کامل در stack_processor_top_tb.v
۳۴	 ۱۰ توضیح کامل آزمون‌های الزامی برنامه نهایی
۳۴	 ۱.۱۰ ورودی $X = 0$
۳۴	 ۲.۱۰ ورودی $X = 31$
۳۴	 ۳.۱۰ ورودی $X = 32$
۳۵	 ۴.۱۰ ورودی $X = 53$
۳۵	 ۵.۱۰ ورودی $X = 54$
۳۵	 ۶.۱۰ ورودی $X = 63$
۳۵	 ۷.۱۰ ورودی $X = 64$
۳۶	 ۸.۱۰ ورودی $X = 127$
۳۶	 ۹.۱۰ ورودی $X = 128$
۳۶	 ۱۰.۱۰ ورودی $X = 255$
۳۷	 ۱۱ شکل موج‌های منتخب و تحلیل آن‌ها
۳۸	 ۱.۱۱ شکل موج $X = 31$
۳۸	 ۲.۱۱ شکل موج $X = 32$
۳۹	 ۳.۱۱ شکل موج $X = 54$
۳۹	 ۴.۱۱ شکل موج $X = 64$
۴۰	 ۵.۱۱ شکل موج $X = 128$
۴۰	 ۱۲ انطباق با الزامات صورت آزمایش
۴۱	 ۱۳ فایل‌های تحویلی
۴۱	 ۱۴ جمع‌بندی

۱ هدف آزمایش

هدف این آزمایش، توسعه و پیاده‌سازی یک پردازنده با معماری پشته‌ای^۱ است که دارای یک پشته با هشت ثبات هشت‌بیتی و حافظه‌ای با ۲۵۶ خانه هشت‌بیتی است. هشت خانه انتهایی فضای نشانی، یعنی F8 تا FF، برای ورودی و خروجی نگاشت شده در حافظه^۲ در نظر گرفته می‌شوند و ارتباط پردازنده با کلیدها، چراغ‌ها و نمایشگرهای هفت‌قسمتی^۳ فقط از طریق همین نشانی‌ها انجام می‌شود.

پردازنده پایه با سه دستور SWAP، SHL و CMP توسعه داده می‌شود و در نتیجه مجموعه نهایی شامل ۱۱ دستور است. برای پشتیبانی از این سه دستور، مسیر داده^۴ و واحد کنترل^۵ توسعه یافته‌اند. در ادامه، با استفاده از همین پردازنده، برنامه‌ای در سطح کد ماشین^۶ نوشته شده است که مقدار هشت‌بیتی X را دریافت کرده و تابع چندبخشی خواسته شده را محاسبه می‌کند.

تابع مورد نظر برابر است با

$$Y = \begin{cases} 3X + 5, & 0 \leq X < 32, \\ 2X + 20, & 32 \leq X < 64, \\ X - 40, & 64 \leq X \leq 127. \end{cases}$$

تشخیص بازه ورودی با دستور CMP و پرش‌های شرطی انجام می‌شود. همچنین برای محاسبه $2X$ از SHL و برای $3X$ از رابطه $3X = X + 2X$ استفاده شده است؛ بنابراین در بخش قابل سنتز طراحی هیچ عملگر ضرب مستقیمی برای پیاده‌سازی تابع وجود ندارد.

۲ مشخصات معماری و قرارداد پشته

۱.۲ ساختار کلی پردازنده

پشته شامل هشت ثبات هشت‌بیتی است و اشاره‌گر پشته به نخستین خانه خالی اشاره می‌کند. مقدار بالاترین عنصر پشته با A و عنصر دوم از بالا با B نمایش داده می‌شود. در پیاده‌سازی حاضر:

$$A = \text{stack}[\text{SP}-1], \quad B = \text{stack}[\text{SP}-2].$$

در دستورهای دو عملوندی ADD و SUB دو عنصر بالا با یک نتیجه جایگزین می‌شوند. مطابق تعریف تفصیلی دستورها، تفریق به صورت

$$B - A$$

انجام می‌شود و همین تفاضل در دستور CMP نیز برای به‌روزرسانی پرچم‌ها به کار می‌رود.

¹Stack Architecture

²Memory-Mapped I/O

³Seven-Segment Display

⁴Datapath

⁵Control Unit

⁶Machine Code

پردازنده دارای دو پرچم S و Z است. Z در صورت صفر بودن نتیجه محاسبه یک می‌شود و S برابر بیت شماره هفت نتیجه هشت‌بیتی است. تنها دستورهای ADD ، SUB و CMP مجاز به تغییر این دو پرچم هستند. تمام محاسبات علامت‌دار و مبنی بر نمایش متمم دو انجام می‌شوند.

۲.۲ مجموعه دستورها

کد عملیاتی^۷ هر دستور در چهار بیت کم‌ارزش ثبات دستور قرار می‌گیرد. در دستورهای پرش، شمارنده برنامه^۸ مقصد اجرای بعدی را نگهداری می‌کند. جدول ۱ مجموعه کامل ۱۱ دستور را نشان می‌دهد.

دستور	کد عملیاتی	عملکرد
PUSHC C	0000	قرار دادن ثابت هشت‌بیتی C روی پشته
PUSH M	0001	خواندن خانه حافظه یا درگاه ورودی M و قرار دادن آن روی پشته
POP M	0010	حذف عنصر بالا و نوشتن آن در حافظه یا درگاه خروجی M
JUMP	0011	حذف نشانی بالای پشته و قرار دادن آن در شمارنده برنامه
JZ	0100	حذف نشانی پرش و پرش در صورت $Z=1$
JS	0101	حذف نشانی پرش و پرش در صورت $S=1$
ADD	0110	جایگزینی A و B با $A + B$ و به‌روزرسانی S, Z
SUB	0111	جایگزینی A و B با $B - A$ و به‌روزرسانی S, Z
SWAP	1000	جابجایی دو عنصر بالای پشته بدون تغییر پرچم‌ها
SHL	1001	انتقال یک‌بیتی عنصر بالای پشته به چپ بدون تغییر پرچم‌ها
CMP	1010	محاسبه موقت $B - A$ ، بدون تغییر پشته، و به‌روزرسانی S, Z

جدول ۱: مجموعه دستورهای پردازنده توسعه‌یافته

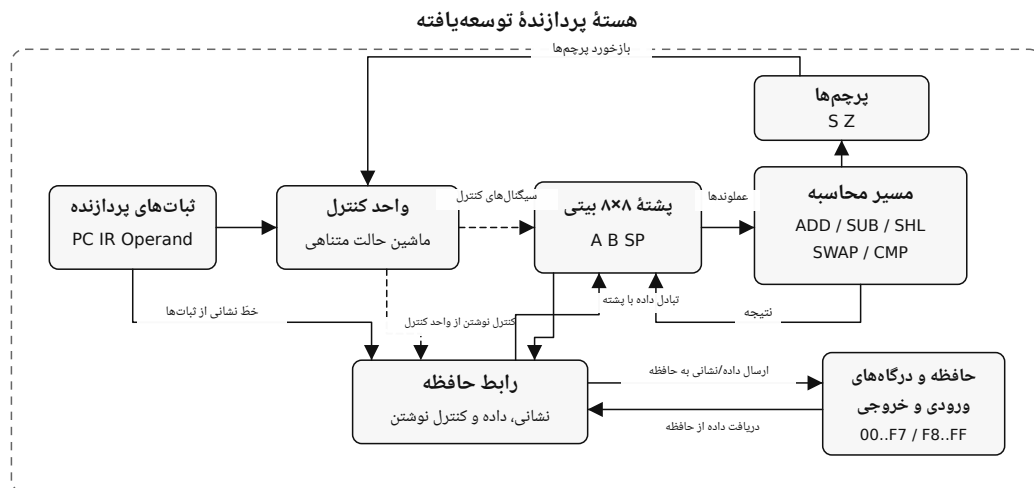
کدهای 1011 تا 1111 رزرو شده‌اند و در این آزمایش استفاده نشده‌اند.

۳.۲ نمودار بلوکی پردازنده توسعه‌یافته

شکل ۱ نمودار بلوکی پردازنده را نشان می‌دهد. حافظه و واحد ورودی و خروجی خارج از هسته قرار دارند و از طریق چهار سیگنال نشانی، داده خواندنی، داده نوشتنی و فعال‌ساز نوشتن با پردازنده ارتباط برقرار می‌کنند.

⁷Opcode

⁸Program Counter



شکل ۱: نمودار بلوکی پردازنده توسعه یافته

۳ توسعه مسیر داده برای سه دستور جدید

۱.۳ دستور SWAP

برای اجرای **SWAP** لازم است هر دو عنصر بالای پشته به طور همزمان در دسترس باشند. در حالت اجرای این دستور، محتوای خانه ۱- **SP** با محتوای ۲- **SP** جابه جا می شود. اشاره گر پشته تغییر نمی کند و هیچ یک از پرچم ها نیز به روزسانی نمی شوند. استفاده از انتساب های غیر مسدودکننده در منطق ترتیبی باعث می شود دو مقدار قدیمی به صورت صحیح با یکدیگر مبادله شوند.

۲.۳ دستور SHL

برای **SHL** یک مسیر انتقال یک بیتی به چپ روی عنصر بالای پشته اضافه شده است:

$$\text{Top} \leftarrow \text{Top} \ll 1.$$

بیت صفر نتیجه جدید صفر می شود و بیت شماره هفت قبلی حذف می گردد. نتیجه هشت بیتی در همان خانه بالای پشته ذخیره می شود؛ بنابراین عمق پشته ثابت می ماند. مطابق صورت آزمایش، این دستور پرچم های **S** و **Z** را تغییر نمی دهد.

۳.۳ دستور CMP

دستور **CMP** از همان مسیر تفریق استفاده می کند، اما نتیجه را در پشته نمی نویسد. مقدار موقت

$$T = B - A$$

محاسبه می شود و فقط پرچم ها از آن ساخته می شوند:

$$Z = (T = 0), \quad S = T_7.$$

ترتیب، عمق و محتوای پشته پس از اجرای CMP بدون تغییر باقی می ماند. همین ویژگی اجازه می دهد دو مقدار برای مقایسه روی پشته قرار گیرند، پرچم ها تعیین شوند و سپس دو عملوند با دو دستور POP پاک شوند.

۴.۳ جدول ریز عملیات دستورهای جدید

دستور	ریز عملیات	تغییر SP	تغییر پرچم ها
SWAP	$stack[SP-1] \leftarrow B, stack[SP-2] \leftarrow A$	بدون تغییر	بدون تغییر
SHL	$stack[SP-1] \leftarrow A \ll 1$	بدون تغییر	بدون تغییر
CMP	$T \leftarrow B - A, Z \leftarrow (T = 0), S \leftarrow T_7$	بدون تغییر	S, Z

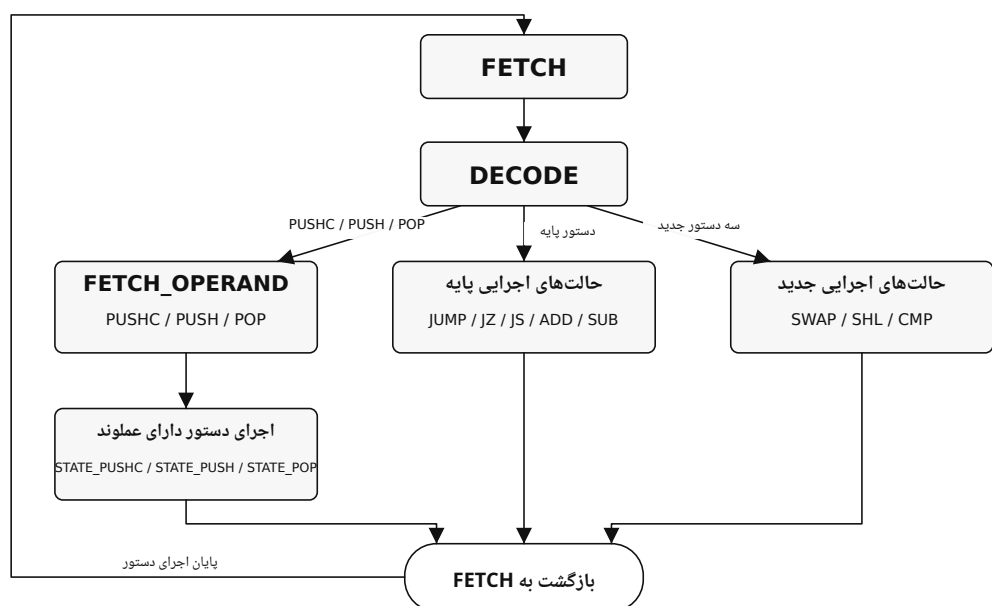
جدول ۲: ریز عملیات سه دستور جدید

۴ توسعه واحد کنترل

واحد کنترل به صورت ماشین حالت متناهی^۹ پیاده سازی شده است. روند عمومی اجرای دستور شامل حالت های FETCH و DECODE است. دستورهای دارای عملوند هشت بیتی، یعنی PUSH، PUSHC و POP، پیش از اجرا وارد حالت FETCH_OPERAND می شوند. برای سه دستور جدید نیز سه حالت اجرایی مستقل STATE_SHL، STATE_SWAP و STATE_CMP به واحد کنترل افزوده شده اند.

در JUMP، نشانی بالای پشته همیشه حذف و در شمارنده برنامه قرار می گیرد. در JZ و JS نیز نشانی پرش در هر دو حالت برقرار یا برقرار نبودن شرط از پشته حذف می شود؛ تنها نوشتن در PC وابسته به پرچم مربوط است. این رفتار باعث می شود پس از یک پرش شرطی ناموفق نیز نشانی مقصد روی پشته باقی نماند.

^۹Finite State Machine



شکل ۲: ساختار کلی گذارهای واحد کنترل و افزوده‌شدن سه حالت جدید

۵ نگاشت حافظه و ورودی و خروجی

حافظه عادی در بازه 00 تا F7 قرار دارد و هشت نشانی انتهایی به درگاه‌های ورودی و خروجی اختصاص یافته‌اند. مقدار نهایی روی دو رقم در مبنای شانزده^{۱۰} نمایش داده می‌شود. جدول ۳ نگاشت مورد استفاده را نشان می‌دهد.

نشانی	نوع دسترسی	عملکرد
F8	فقط خواندنی	ورودی هشت‌بیتی X از کلیدهای بورد
F9	فقط نوشتنی	رقم کم‌ارزش خروجی در مبنای شانزده؛ فقط چهار بیت کم‌ارزش داده نوشته شده استفاده می‌شود
FA	فقط نوشتنی	رقم پرارزش خروجی در مبنای شانزده؛ فقط چهار بیت کم‌ارزش داده نوشته شده استفاده می‌شود
FB	فقط نوشتنی	بیت صفر، خروجی چراغ خطا
FC	فقط نوشتنی	بیت صفر، خروجی چراغ پایان محاسبه
FD - - FF	رزروده	در این آزمایش استفاده نمی‌شوند

جدول ۳: نگاشت ورودی و خروجی در فضای نشانی

در حالت موفق، ابتدا خطا صفر می‌شود، رقم کم‌ارزش در F9 و رقم پرارزش در FA نوشته می‌شوند و در پایان مقدار یک در FC قرار می‌گیرد. در حالت خطا، مقدار یک در FB نوشته شده، هر دو رقم نمایشگر با نوشتن صفر در F9 و FA پاک

¹⁰Hexadecimal

می‌شوند و سپس FC یک می‌شود. بنابراین یک‌شدن چراغ پایان به معنای معتبرشدن وضعیت نهایی خروجی‌هاست.

۶ ساختار فایل‌های اصلی و کدهای قابل سنتز

۱.۶ هسته پردازنده: `stack_cpu.v`

این فایل پشته، شمارنده برنامه، ثبات دستور، ثبات عملوند، پرچم‌ها، رابط حافظه و واحد کنترل را پیاده‌سازی می‌کند. تمام ۱۱ دستور در همین پیمان‌ه اجرا می‌شوند.

```
1 module stack_cpu (
2     input wire      clk,
3     input wire      reset,
4
5     output reg [7:0] memory_address,
6     input wire [7:0] memory_read_data,
7     output reg [7:0] memory_write_data,
8     output reg      memory_write_enable
9 );
10
11 // -----
12 // Opcodes
13 // -----
14
15 localparam [3:0] OP_PUSHC = 4'b0000;
16 localparam [3:0] OP_PUSH  = 4'b0001;
17 localparam [3:0] OP_POP   = 4'b0010;
18 localparam [3:0] OP_JUMP  = 4'b0011;
19 localparam [3:0] OP_JZ    = 4'b0100;
20 localparam [3:0] OP_JS    = 4'b0101;
21 localparam [3:0] OP_ADD   = 4'b0110;
22 localparam [3:0] OP_SUB   = 4'b0111;
23 localparam [3:0] OP_SWAP  = 4'b1000;
24 localparam [3:0] OP_SHL   = 4'b1001;
25 localparam [3:0] OP_CMP   = 4'b1010;
26
27
28 // -----
29 // FSM states
30 // -----
31
32 localparam [3:0] STATE_FETCH      = 4'd0;
33 localparam [3:0] STATE_DECODE    = 4'd1;
34 localparam [3:0] STATE_FETCH_OPERAND = 4'd2;
35 localparam [3:0] STATE_PUSHC     = 4'd3;
36 localparam [3:0] STATE_PUSH      = 4'd4;
37 localparam [3:0] STATE_POP       = 4'd5;
```

```

38  localparam [3:0] STATE_JUMP          = 4'd6;
39  localparam [3:0] STATE_JZ           = 4'd7;
40  localparam [3:0] STATE_JS           = 4'd8;
41  localparam [3:0] STATE_ADD           = 4'd9;
42  localparam [3:0] STATE_SUB           = 4'd10;
43  localparam [3:0] STATE_SWAP          = 4'd11;
44  localparam [3:0] STATE_SHL           = 4'd12;
45  localparam [3:0] STATE_CMP           = 4'd13;
46
47
48  // -----
49  // CPU registers
50  // -----
51
52  reg [7:0] program_counter;
53  reg [7:0] instruction_register;
54  reg [7:0] operand_register;
55
56  reg [7:0] stack [0:7];
57  reg [3:0] stack_pointer;
58
59  reg zero_flag;
60  reg sign_flag;
61
62  reg [3:0] state;
63
64  integer i;
65
66
67  // -----
68  // Datapath signals
69  // -----
70
71  wire [3:0] opcode;
72
73  wire [7:0] stack_top;
74  wire [7:0] stack_second;
75
76  wire [7:0] add_result;
77  wire [7:0] sub_result;
78
79
80  assign opcode = instruction_register[3:0];
81
82  // Stack pointer points to the next free location
83  assign stack_top =
84      (stack_pointer > 0)

```

```

85     ? stack[stack_pointer - 4'd1]
86     : 8'd0;
87
88     assign stack_second =
89         (stack_pointer > 1)
90         ? stack[stack_pointer - 4'd2]
91         : 8'd0;
92
93     assign add_result = stack_second + stack_top;
94
95     // SUB and CMP use B - A
96     assign sub_result = stack_second - stack_top;
97
98
99     // -----
100    // Memory interface
101    // -----
102
103    always @(*) begin
104
105        // Fetch instructions by default
106        memory_address      = program_counter;
107        memory_write_data   = 8'd0;
108        memory_write_enable = 1'b0;
109
110        case (state)
111
112            STATE_PUSH: begin
113                // Read data from memory or input port
114                memory_address = operand_register;
115            end
116
117            STATE_POP: begin
118                // Write stack top to memory or output port
119                memory_address      = operand_register;
120                memory_write_data   = stack_top;
121                memory_write_enable = 1'b1;
122            end
123
124            default: begin
125                memory_address = program_counter;
126            end
127
128        endcase
129    end
130
131

```

```

132 // -----
133 // Control unit
134 // -----
135
136 always @(posedge clk or posedge reset) begin
137
138     if (reset) begin
139
140         program_counter    <= 8'd0;
141         instruction_register <= 8'd0;
142         operand_register    <= 8'd0;
143
144         stack_pointer <= 4'd0;
145
146         zero_flag <= 1'b0;
147         sign_flag <= 1'b0;
148
149         state <= STATE_FETCH;
150
151         // Clear stack registers
152         for (i = 0; i < 8; i = i + 1)
153             stack[i] <= 8'd0;
154
155     end
156
157     else begin
158
159         case (state)
160
161             // -----
162             // Fetch instruction
163             // -----
164
165             STATE_FETCH: begin
166
167                 instruction_register <= memory_read_data;
168                 program_counter <= program_counter + 8'd1;
169
170                 state <= STATE_DECODE;
171
172             end
173
174
175             // -----
176             // Decode instruction
177             // -----
178

```

```

179         STATE_DECODE: begin
180
181             case (opcode)
182
183                 OP_PUSHC,
184                 OP_PUSH,
185                 OP_POP:
186                     state <= STATE_FETCH_OPERAND;
187
188                 OP_JUMP:
189                     state <= STATE_JUMP;
190
191                 OP_JZ:
192                     state <= STATE_JZ;
193
194                 OP_JS:
195                     state <= STATE_JS;
196
197                 OP_ADD:
198                     state <= STATE_ADD;
199
200                 OP_SUB:
201                     state <= STATE_SUB;
202
203                 OP_SWAP:
204                     state <= STATE_SWAP;
205
206                 OP_SHL:
207                     state <= STATE_SHL;
208
209                 OP_CMP:
210                     state <= STATE_CMP;
211
212                 default:
213                     state <= STATE_FETCH;
214
215             endcase
216
217         end
218
219         // -----
220         // Fetch instruction operand
221         // -----
222
223         STATE_FETCH_OPERAND: begin
224
225

```

```

226         operand_register <= memory_read_data;
227         program_counter <= program_counter + 8'd1;
228
229         case (opcode)
230
231             OP_PUSHC:
232                 state <= STATE_PUSHC;
233
234             OP_PUSH:
235                 state <= STATE_PUSH;
236
237             OP_POP:
238                 state <= STATE_POP;
239
240             default:
241                 state <= STATE_FETCH;
242
243         endcase
244
245     end
246
247
248     // -----
249     // PUSHC
250     // -----
251
252     STATE_PUSHC: begin
253
254         stack[stack_pointer] <= operand_register;
255         stack_pointer <= stack_pointer + 4'd1;
256
257         state <= STATE_FETCH;
258
259     end
260
261
262     // -----
263     // PUSH
264     // -----
265
266     STATE_PUSH: begin
267
268         stack[stack_pointer] <= memory_read_data;
269         stack_pointer <= stack_pointer + 4'd1;
270
271         state <= STATE_FETCH;
272

```

```

273         end
274
275         // -----
276         // POP
277         // -----
278
279         STATE_POP: begin
280
281             // Memory write occurs through combinational interface
282             stack_pointer <= stack_pointer - 4'd1;
283
284
285             state <= STATE_FETCH;
286
287         end
288
289         // -----
290         // JUMP
291         // -----
292
293         STATE_JUMP: begin
294
295             program_counter <= stack_top;
296             stack_pointer <= stack_pointer - 4'd1;
297
298
299             state <= STATE_FETCH;
300
301         end
302
303         // -----
304         // JZ
305         // -----
306
307         STATE_JZ: begin
308
309             // Jump target is always removed from stack
310             if (zero_flag)
311                 program_counter <= stack_top;
312
313
314             stack_pointer <= stack_pointer - 4'd1;
315
316             state <= STATE_FETCH;
317
318         end
319

```

```

320
321 // -----
322 // JS
323 // -----
324
325 STATE_JS: begin
326
327     // Jump target is always removed from stack
328     if (sign_flag)
329         program_counter <= stack_top;
330
331     stack_pointer <= stack_pointer - 4'd1;
332
333     state <= STATE_FETCH;
334
335 end
336
337
338 // -----
339 // ADD
340 // -----
341
342 STATE_ADD: begin
343
344     // Replace B and A with B + A
345     stack[stack_pointer - 4'd2] <= add_result;
346     stack_pointer <= stack_pointer - 4'd1;
347
348     // Update status flags
349     zero_flag <= (add_result == 8'd0);
350     sign_flag <= add_result[7];
351
352     state <= STATE_FETCH;
353
354 end
355
356
357 // -----
358 // SUB
359 // -----
360
361 STATE_SUB: begin
362
363     // Replace B and A with B - A
364     stack[stack_pointer - 4'd2] <= sub_result;
365     stack_pointer <= stack_pointer - 4'd1;
366

```

```

367         // Update status flags
368         zero_flag <= (sub_result == 8'd0);
369         sign_flag <= sub_result[7];
370
371         state <= STATE_FETCH;
372
373     end
374
375
376     // -----
377     // SWAP
378     // -----
379
380     STATE_SWAP: begin
381
382         // Exchange the top two stack elements
383         stack[stack_pointer - 4'd1]
384             <= stack[stack_pointer - 4'd2];
385
386         stack[stack_pointer - 4'd2]
387             <= stack[stack_pointer - 4'd1];
388
389         state <= STATE_FETCH;
390
391     end
392
393
394     // -----
395     // SHL
396     // -----
397
398     STATE_SHL: begin
399
400         // Shift stack top left by one bit
401         stack[stack_pointer - 4'd1]
402             <= stack_top << 1;
403
404         state <= STATE_FETCH;
405
406     end
407
408
409     // -----
410     // CMP
411     // -----
412
413     STATE_CMP: begin

```

```

414
415         // Compare B and A without modifying stack
416         zero_flag <= (sub_result == 8'd0);
417         sign_flag <= sub_result[7];
418
419         state <= STATE_FETCH;
420
421     end
422
423
424     // -----
425     // Recovery
426     // -----
427
428     default: begin
429         state <= STATE_FETCH;
430     end
431
432 endcase
433
434 end
435
436 end
437
438 endmodule

```

کد ۱: پیاده‌سازی هسته پردازنده در stack_cpu.v

۲.۶ حافظه و درگاه‌ها: memory_io.v

این پیمان‌نامه حافظه برنامه و داده را همراه با نگاشت نشانی‌های F8 تا FC پیاده‌سازی می‌کند. محتوای program.hex در آغاز شبیه‌سازی/سنتز در حافظه بارگذاری می‌شود. درگاه F8 فقط خواندنی است و چهار خروجی دیگر مطابق جدول ۳ فقط نوشتنی هستند.

```

1 module memory_io (
2     input wire      clk,
3     input wire      reset,
4
5     input wire [7:0] address,
6     input wire [7:0] write_data,
7     input wire      write_enable,
8     output reg  [7:0] read_data,
9
10    input wire [7:0] input_x,
11

```

```

12  output reg [3:0] hex_low,
13  output reg [3:0] hex_high,
14  output reg      error_led,
15  output reg      done_led
16 );
17
18  // -----
19  // Memory-mapped I/O addresses
20  // -----
21
22  localparam [7:0] ADDR_INPUT_X  = 8'hF8;
23  localparam [7:0] ADDR_HEX_LOW  = 8'hF9;
24  localparam [7:0] ADDR_HEX_HIGH = 8'hFA;
25  localparam [7:0] ADDR_ERROR    = 8'hFB;
26  localparam [7:0] ADDR_DONE     = 8'hFC;
27
28
29  // -----
30  // Program and data memory
31  // -----
32
33  reg [7:0] memory [0:247];
34
35  integer i;
36
37
38  // -----
39  // Memory initialization
40  // -----
41
42  initial begin
43
44      // Clear memory
45      for (i = 0; i < 248; i = i + 1)
46          memory[i] = 8'h00;
47
48      // Load machine-code program
49      $readmemh("program.hex", memory);
50
51      hex_low  = 4'h0;
52      hex_high = 4'h0;
53      error_led = 1'b0;
54      done_led  = 1'b0;
55
56  end
57
58

```

```

59 // -----
60 // Combinational read
61 // -----
62
63 always @(*) begin
64
65     case (address)
66
67         ADDR_INPUT_X: begin
68             // Read input switches
69             read_data = input_x;
70         end
71
72         ADDR_HEX_LOW,
73         ADDR_HEX_HIGH,
74         ADDR_ERROR,
75         ADDR_DONE,
76         8'hFD,
77         8'hFE,
78         8'hFF: begin
79             // Write-only or reserved ports
80             read_data = 8'h00;
81         end
82
83         default: begin
84
85             if (address < 8'hF8)
86                 read_data = memory[address];
87             else
88                 read_data = 8'h00;
89
90         end
91
92     endcase
93
94 end
95
96
97 // -----
98 // Synchronous write
99 // -----
100
101 always @(posedge clk or posedge reset) begin
102
103     if (reset) begin
104
105         // Reset output ports only

```

```

106         hex_low    <= 4'h0;
107         hex_high    <= 4'h0;
108         error_led    <= 1'b0;
109         done_led     <= 1'b0;
110
111     end
112
113     else if (write_enable) begin
114
115         case (address)
116
117             ADDR_HEX_LOW: begin
118                 // Store low hexadecimal digit
119                 hex_low <= write_data[3:0];
120             end
121
122             ADDR_HEX_HIGH: begin
123                 // Store high hexadecimal digit
124                 hex_high <= write_data[3:0];
125             end
126
127             ADDR_ERROR: begin
128                 // Update error LED
129                 error_led <= write_data[0];
130             end
131
132             ADDR_DONE: begin
133                 // Update done LED
134                 done_led <= write_data[0];
135             end
136
137             ADDR_INPUT_X,
138             8'hFD,
139             8'hFE,
140             8'hFF: begin
141                 // Ignore writes to read-only or reserved addresses
142             end
143
144             default: begin
145
146                 if (address < 8'hF8)
147                     memory[address] <= write_data;
148
149             end
150
151         endcase
152

```

```

153         end
154
155     end
156
157 endmodule

```

کد ۲: پیاده‌سازی حافظه و ورودی و خروجی نگاشت‌شده در حافظه

۳.۶ سامانه پردازنده: processor_system.v

این فایل نقش اتصال میان هسته پردازنده و حافظه/درگاه‌ها را دارد. سیگنال‌های memory_read_data، memory_address، memory_write_data و memory_write_enable میان دو پیمانه ردوبدل می‌شوند و هیچ منطق محاسباتی اضافه‌ای در این سطح قرار نگرفته است.

```

1 module processor_system (
2     input wire      clk,
3     input wire      reset,
4
5     input wire [7:0] input_x,
6
7     output wire [3:0] hex_low,
8     output wire [3:0] hex_high,
9     output wire      error_led,
10    output wire      done_led
11 );
12
13 // -----
14 // CPU-memory interface
15 // -----
16
17 wire [7:0] memory_address;
18 wire [7:0] memory_read_data;
19 wire [7:0] memory_write_data;
20 wire      memory_write_enable;
21
22
23 // -----
24 // CPU core
25 // -----
26
27 stack_cpu cpu (
28     .clk          (clk),
29     .reset        (reset),
30
31     .memory_address (memory_address),

```

```

32     .memory_read_data    (memory_read_data),
33     .memory_write_data   (memory_write_data),
34     .memory_write_enable (memory_write_enable)
35 );
36
37
38 // -----
39 // Memory and memory-mapped I/O
40 // -----
41
42 memory_io memory_unit (
43     .clk          (clk),
44     .reset        (reset),
45
46     .address      (memory_address),
47     .write_data   (memory_write_data),
48     .write_enable (memory_write_enable),
49     .read_data    (memory_read_data),
50
51     .input_x      (input_x),
52
53     .hex_low      (hex_low),
54     .hex_high     (hex_high),
55     .error_led    (error_led),
56     .done_led     (done_led)
57 );
58
59 endmodule

```

کد ۳: اتصال پردازنده به حافظه و درگاه‌ها

۴.۶ رمزگشای نمایشگر هفت‌قسمتی: `seven_segment_decoder.v`

هر رقم خروجی چهار بیت است و باید ارقام ۰ تا F را نمایش دهد. رمزگشا خروجی فعال‌پایین هفت‌بیتی تولید می‌کند؛ یعنی صفر منطقی باعث روشن شدن قسمت متناظر نمایشگر می‌شود.

```

1 module seven_segment_decoder (
2     input wire [3:0] digit,
3     output reg  [6:0] segments
4 );
5
6 // -----
7 // Hexadecimal seven-segment decoder
8 // Active-low outputs
9 // -----

```

```

10
11  always @(*) begin
12
13      case (digit)
14
15          4'h0: segments = 7'b1000000;
16          4'h1: segments = 7'b1111001;
17          4'h2: segments = 7'b0100100;
18          4'h3: segments = 7'b0110000;
19          4'h4: segments = 7'b0011001;
20          4'h5: segments = 7'b0010010;
21          4'h6: segments = 7'b0000010;
22          4'h7: segments = 7'b1111000;
23          4'h8: segments = 7'b0000000;
24          4'h9: segments = 7'b0010000;
25
26          4'hA: segments = 7'b0001000;
27          4'hB: segments = 7'b0000011;
28          4'hC: segments = 7'b1000110;
29          4'hD: segments = 7'b0100001;
30          4'hE: segments = 7'b0000110;
31          4'hF: segments = 7'b0001110;
32
33          default: segments = 7'b1111111;
34
35      endcase
36
37  end
38
39 endmodule

```

کد ۴: رمزگشای مبنای شانزده برای نمایشگر هفت‌قسمتی

۵.۶ پیمانه سطح بالا: stack_processor_top.v

این پیمانه processor_system.v را به دو رمزگشای نمایشگر متصل می‌کند. دو سیگنال چهاربیتی داخلی برای رقم پرارزش و کم‌ارزش ایجاد می‌شوند و خروجی‌های هفت‌بیتی نهایی، همراه با error_led و done_led، رابط سطح بالای طراحی را تشکیل می‌دهند.

```

1 module stack_processor_top (
2     input wire    clk,
3     input wire    reset,
4
5     input wire [7:0] input_x,
6

```

```

7  output wire [6:0] seven_segment_low,
8  output wire [6:0] seven_segment_high,
9
10 output wire      error_led,
11 output wire      done_led
12 );
13
14 // -----
15 // Internal hexadecimal digits
16 // -----
17
18 wire [3:0] hex_low;
19 wire [3:0] hex_high;
20
21
22 // -----
23 // Processor system
24 // -----
25
26 processor_system processor (
27     .clk      (clk),
28     .reset     (reset),
29
30     .input_x   (input_x),
31
32     .hex_low   (hex_low),
33     .hex_high  (hex_high),
34     .error_led (error_led),
35     .done_led  (done_led)
36 );
37
38
39 // -----
40 // Low hexadecimal digit
41 // -----
42
43 seven_segment_decoder low_decoder (
44     .digit      (hex_low),
45     .segments   (seven_segment_low)
46 );
47
48
49 // -----
50 // High hexadecimal digit
51 // -----
52
53 seven_segment_decoder high_decoder (

```

```

54     .digit    (hex_high),
55     .segments (seven_segment_high)
56 );
57
58 endmodule

```

کد ۵: پیمانه سطح بالای پردازنده و دو نمایشگر

۷ برنامه زبان ماشین

۱.۷ خانه‌های موقت حافظه

برای نگهداری مقادیر میانی از چهار خانه عادی حافظه استفاده شده است:

نشانی	کاربرد
E0	نگهداری ورودی X
E1	نگهداری Y و سپس باقیمانده تقسیم بر ۱۶ در مرحله تولید رقم کم‌ارزش
E2	شمارنده دفعات کم‌کردن ۱۶ و در نتیجه رقم پرارزش مبنای شانزده
E3	خانه موقت برای دورریختن عملوندهای باقی‌مانده پس از CMP

جدول ۴: خانه‌های موقت مورد استفاده در برنامه

۲.۷ منطق محاسبه تابع

پس از پاک‌کردن خروجی‌های قبلی، X از F8 خوانده و در E0 ذخیره می‌شود. ابتدا علامت ورودی با مقایسه X و صفر بررسی می‌شود. سپس به ترتیب مقایسه با ۳۲ و ۶۴ انجام می‌شود تا یکی از سه بازه تابع انتخاب گردد. در بازه اول، مقدار $2X$ را تولید کرده و با یک بار جمع با X ، مقدار $3X$ ساخته می‌شود. در بازه دوم نیز $2X$ با SHL ساخته شده و سپس ۲۰ به آن افزوده می‌شود. اگر حاصل بزرگ‌تر از ۱۲۷ باشد، بیت علامت نتیجه هشت‌بیتی یک می‌شود و JS برنامه را به مسیر خطا می‌برد. در بازه سوم نیز SUB مقدار $40 - X$ را محاسبه می‌کند.

۳.۷ تبدیل خروجی به دو رقم مبنای شانزده

مجموعه دستورها فاقد انتقال به راست و عملگر و بیتی^{۱۱} است؛ بنابراین رقم پرارزش مستقیماً با انتقال یا پوشاندن بیتی قابل استخراج نیست. برای حل این محدودیت، پس از محاسبه Y ، مقدار آن در E1 و شمارنده رقم پرارزش در E2 نگهداری می‌شود.

^{۱۱}Bitwise AND

تا زمانی که $E1 \geq 16$ باشد، عدد ۱۶ از $E1$ کم و $E2$ یک واحد زیاد می‌شود. در پایان:

$$\text{low} = E1 = Y \bmod 16, \quad \text{high} = E2 = \left\lfloor \frac{Y}{16} \right\rfloor.$$

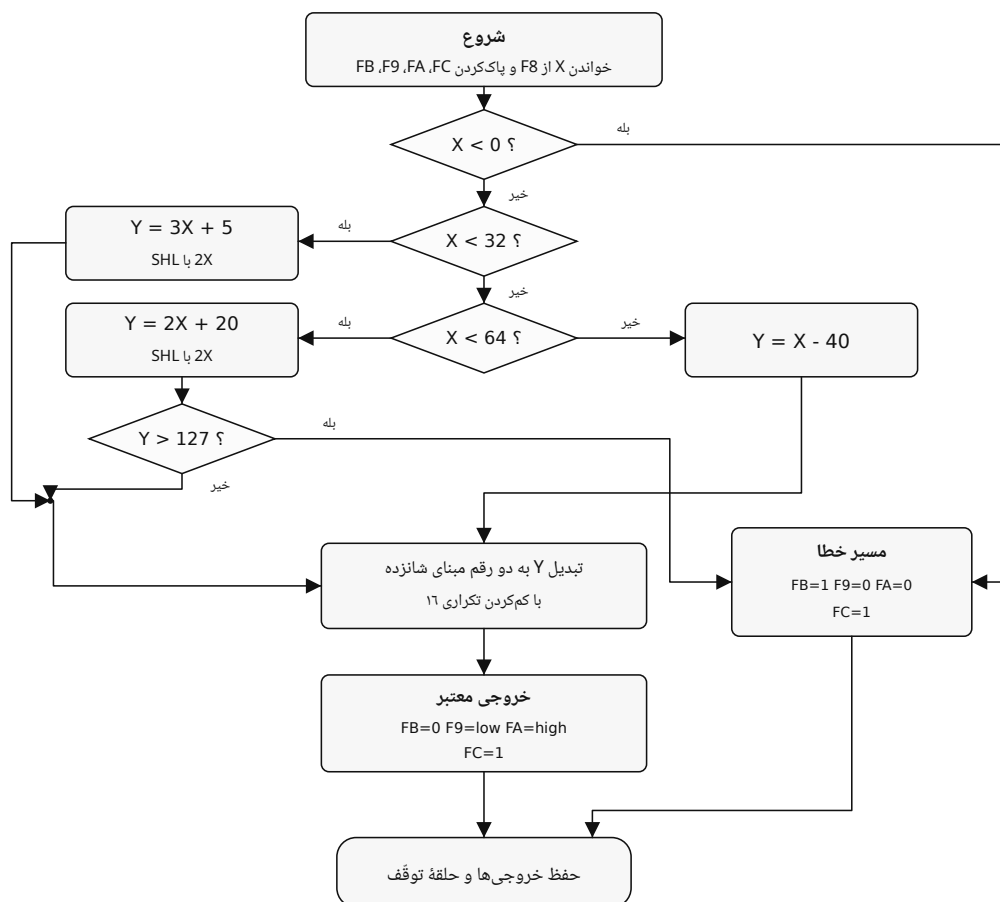
این دو مقدار به ترتیب در $F9$ و FA نوشته می‌شوند. این روش بدون افزودن دستور جدید و فقط با استفاده از دستورهای مجاز پردازنده انجام می‌شود.

۴.۷ حداکثر عمق پشته

حداکثر عمق پشته در تمام مسیرهای برنامه برابر [2] است. در مقایسه‌ها ابتدا مقدار X یا باقیمانده و سپس ثابت مقایسه روی پشته قرار می‌گیرند؛ بنابراین عمق به دو می‌رسد. CMP پشته را تغییر نمی‌دهد و پس از آن دو POP هر دو عملوند را حذف می‌کنند. در محاسبات نیز بیشترین حالت زمانی رخ می‌دهد که دو عملوند برای ADD یا SUB هم‌زمان روی پشته قرار گرفته‌اند. در حلقه تبدیل مبنای شانزده نیز مقایسه و تفریق هرگز بیش از دو عنصر هم‌زمان ایجاد نمی‌کنند. نشانی پرش نیز زمانی روی پشته قرار می‌گیرد که عملوندهای قبلی پاک شده‌اند. در نتیجه هیچ مسیر اجرایی به عمق سه یا بیشتر نمی‌رسد.

۵.۷ نمودار روند برنامه

نمودار روند^{۱۲} کامل برنامه در شکل ۳ آمده است.



شکل ۳: نمودار روند برنامه محاسبه تابع چندبخشی

۶.۷ جدول کامل کد ماشین

جدول ۵ تمام دستورهای اجرایی برنامه `program.hex` از نشانی ۰۰ تا AA را نشان می‌دهد. خانه‌های AB تا F7 برای تکمیل حافظه عادی با ۰۰ پر شده‌اند و به دلیل حلقه توقف در AA - A8 هرگز اجرا نمی‌شوند.

جدول ۵: محتوای کامل بخش اجرایی حافظه برنامه

نشانی	کد ماشین	دستور	توضیح
۰۰	۰۰ ۰۰	PUSHC ۰۰	صفرکردن مقدار اولیه خطا
۰۲	۰۲ FB	POP FB	نوشتن صفر در درگاه خطا
۰۴	۰۰ ۰۰	PUSHC ۰۰	آماده‌سازی صفر برای رقم کم‌ارزش
۰۶	۰۲ F9	POP F9	پاک‌کردن رقم کم‌ارزش نمایشگر

^{۱۲}Flowchart

نشانی	کد ماشین	دستور	توضیح
08	00 00	PUSHC 00	آماده‌سازی صفر برای رقم پرارزش
0A	02 FA	POP FA	پاک‌کردن رقم پرارزش نمایشگر
0C	00 00	PUSHC 00	آماده‌سازی صفر برای پایان
0E	02 FC	POP FC	صفرکردن چراغ پایان
10	01 F8	PUSH F8	خواندن ورودی X
12	02 E0	POP E0	ذخیره X در خانه موقت E0
14	01 E0	PUSH E0	قرار دادن X برای مقایسه با صفر
16	00 00	PUSHC 00	قرار دادن صفر روی پشته
18	0A	CMP	محاسبه موقت X-0 و به‌روزرسانی پرچم‌ها
19	02 E3	POP E3	حذف عملوند اول مقایسه
1B	02 E3	POP E3	حذف عملوند دوم مقایسه
1D	00 95	PUSHC 95	قرار دادن نشانی مسیر خطا
1F	05	JS	پرش به خطا در صورت منفی بودن X
20	01 E0	PUSH E0	قرار دادن X برای مقایسه با ۳۲
22	00 20	PUSHC 20	قرار دادن ۳۲ روی پشته
24	0A	CMP	مقایسه X با ۳۲
25	02 E3	POP E3	پاک‌سازی عملوند مقایسه
27	02 E3	POP E3	پاک‌سازی عملوند مقایسه
29	00 3B	PUSHC 3B	قرار دادن نشانی بازه اول
2B	05	JS	اگر $X < 32$ باشد، پرش به بازه اول
2C	01 E0	PUSH E0	قرار دادن X برای مقایسه با ۶۴
2E	00 40	PUSHC 40	قرار دادن ۶۴ روی پشته
30	0A	CMP	مقایسه X با ۶۴
31	02 E3	POP E3	پاک‌سازی عملوند مقایسه
33	02 E3	POP E3	پاک‌سازی عملوند مقایسه
35	00 49	PUSHC 49	قرار دادن نشانی بازه دوم
37	05	JS	اگر $X < 64$ باشد، پرش به بازه دوم
38	00 57	PUSHC 57	قرار دادن نشانی بازه سوم
3A	03	JUMP	پرش بدون شرط به بازه سوم
3B	01 E0	PUSH E0	خواندن X برای بازه اول
3D	09	SHL	محاسبه $2X$ با انتقال یک‌بیتی به چپ
3E	01 E0	PUSH E0	قرار دادن X روی پشته
40	06	ADD	محاسبه $X + 2X = 3X$
41	00 05	PUSHC 05	قرار دادن ثابت ۵
43	06	ADD	محاسبه $Y = 3X + 5$
44	02 E1	POP E1	ذخیره Y در E1
46	00 61	PUSHC 61	قرار دادن نشانی بخش تبدیل خروجی
48	03	JUMP	پرش به بخش تبدیل مبنای شانزده
49	01 E0	PUSH E0	خواندن X برای بازه دوم
4B	09	SHL	محاسبه $2X$ با SHL
4C	00 14	PUSHC 14	قرار دادن ثابت ۲۰ ده‌دهی
4E	06	ADD	محاسبه $Y = 2X + 20$ و تعیین پرچم علامت

نشانی	کد ماشین	دستور	توضیح
4F	02 E1	POP E1	ذخیره Y بدون تغییر پرچم‌ها
51	00 95	PUSHC 95	قرار دادن نشانی خطا
53	05	JS	تشخیص $Y > 127$ از بیت علامت و پرش به خطا
54	00 61	PUSHC 61	قرار دادن نشانی بخش تبدیل خروجی
56	03	JUMP	پرش به بخش تبدیل مبنای شانزده
57	01 E0	PUSH E0	خواندن X برای بازه سوّم
59	00 28	PUSHC 28	قرار دادن ثابت ۴۰ ده‌دهی
5B	07	SUB	محاسبه $Y = X - 40$ بر پایه B-A
5C	02 E1	POP E1	ذخیره Y در E1
5E	00 61	PUSHC 61	قرار دادن نشانی بخش تبدیل خروجی
60	03	JUMP	پرش به بخش تبدیل مبنای شانزده
61	00 00	PUSHC 00	مقداردهی اولیه رقم پرارزش
63	02 E2	POP E2	$E2 = 0$
65	01 E1	PUSH E1	قرار دادن باقیمانده جاری روی پشته
67	00 10	PUSHC 10	قرار دادن ۱۶ ده‌دهی
69	0A	CMP	مقایسه باقیمانده با ۱۶
6A	02 E3	POP E3	پاک‌سازی عملوند مقایسه
6C	02 E3	POP E3	پاک‌سازی عملوند مقایسه
6E	00 82	PUSHC 82	قرار دادن نشانی پایان تبدیل
70	05	JS	اگر باقیمانده > 16 باشد، پایان حلقه
71	01 E1	PUSH E1	خواندن باقیمانده جاری
73	00 10	PUSHC 10	قرار دادن ۱۶
75	07	SUB	کم کردن ۱۶ از باقیمانده
76	02 E1	POP E1	ذخیره باقیمانده جدید
78	01 E2	PUSH E2	خواندن رقم پرارزش جاری
7A	00 01	PUSHC 01	قرار دادن ثابت یک
7C	06	ADD	افزایش رقم پرارزش
7D	02 E2	POP E2	ذخیره رقم پرارزش جدید
7F	00 65	PUSHC 65	قرار دادن نشانی ابتدای حلقه
81	03	JUMP	تکرار حلقه تبدیل
82	00 00	PUSHC 00	آماده‌سازی صفر برای خطا
84	02 FB	POP FB	خاموش کردن چراغ خطا
86	01 E1	PUSH E1	خواندن رقم کم‌ارزش
88	02 F9	POP F9	نوشتن رقم کم‌ارزش در F9
8A	01 E2	PUSH E2	خواندن رقم پرارزش
8C	02 FA	POP FA	نوشتن رقم پرارزش در FA
8E	00 01	PUSHC 01	آماده‌سازی علامت پایان
90	02 FC	POP FC	روشن کردن چراغ پایان
92	00 A8	PUSHC A8	قرار دادن نشانی توقف
94	03	JUMP	ورود به حلقه توقف
95	00 01	PUSHC 01	آماده‌سازی مقدار خطا
97	02 FB	POP FB	روشن کردن چراغ خطا

نشانی	کد ماشین	دستور	توضیح
99	00 00	PUSHC 00	آماده‌سازی صفر برای نمایشگر
9B	02 F9	POP F9	پاک‌کردن رقم کم‌ارزش
9D	00 00	PUSHC 00	آماده‌سازی صفر برای نمایشگر
9F	02 FA	POP FA	پاک‌کردن رقم پرارزش
A1	00 01	PUSHC 01	آماده‌سازی علامت پایان
A3	02 FC	POP FC	روشن‌کردن چراغ پایان
A5	00 A8	PUSHC A8	قرار دادن نشانی توقف
A7	03	JUMP	ورود به حلقه توقف
A8	00 A8	PUSHC A8	قرار دادن نشانی خود حلقه
AA	03	JUMP	پرش به A8 و نگه‌داشتن خروجی‌ها

۸ سنتز و مشاهده ساختار در Quartus

۱.۸ روند انجام کار در Quartus

فایل‌های `seven_segment_decoder.v`، `processor_system.v`، `memory_io.v`، `stack_cpu.v` و `stack_processor_top.v` به پروژه افزوده شدند و `stack_processor_top.v` به‌عنوان موجودیت سطح بالا انتخاب شد. برای کامپایل کامل پروژه از مسیر زیر استفاده شد:

Processing > Start Compilation

پس از پایان کامپایل، برای مشاهده نمای سطح انتقال ثبات^{۱۳} مسیر زیر در Quartus استفاده شد:

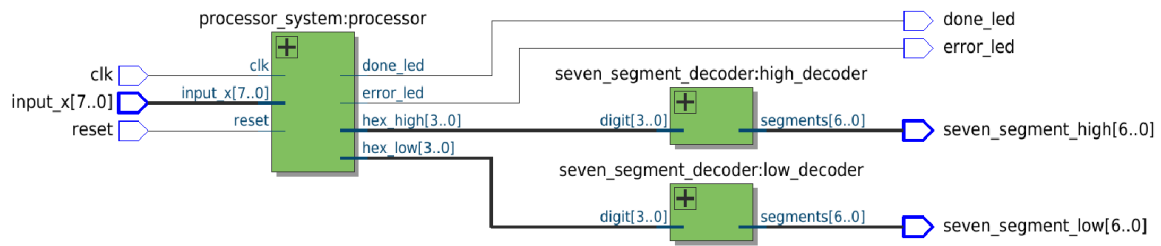
Tools > Netlist Viewers > RTL Viewer

نمای سطح انتقال ثبات حاصل از سنتز^{۱۴}، ارتباط واقعی پیمانه‌ها و منطق استخراج‌شده از توصیف Verilog را نمایش می‌دهد.

¹³Register-Transfer Level (RTL)

¹⁴Synthesis

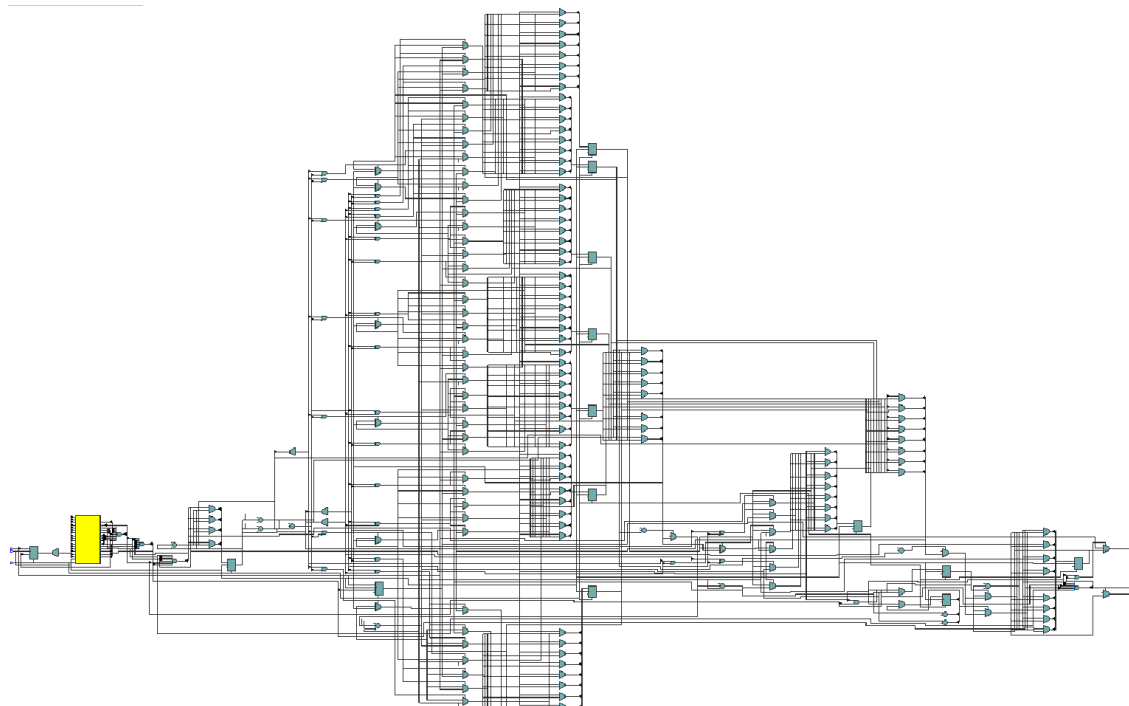
۲.۸ نمای سطح بالای سامانه



شکل ۴: نمای RTL سطح بالای stack_processor_top.v

در شکل ۴ پیمانه processor_system در سمت چپ و دو رمزگشای نمایشگر در سمت راست دیده می‌شوند. دو رقم داخلی hex_high و hex_low از سامانه پردازنده خارج شده و به رمزگشاهای می‌روند؛ درحالی‌که error_led و done_led مستقیماً به خروجی سطح بالا متصل هستند.

۳.۸ نمای تفصیلی پردازنده



شکل ۵: نمای RTL تفصیلی پردازنده پس از سنتز

شکل ۵ نمای تفصیلی منطق سنتز شده را نشان می‌دهد. شلوغی این نما طبیعی است؛ زیرا پشته، ماشین حالت، منطق نشانی‌دهی حافظه، جمع و تفریق، مسیرهای دستورهای جدید و ثبات‌های وضعیت همگی به عناصر منطقی صریح تبدیل شده‌اند. این تصویر در کنار نمودار بلوکی شکل ۱ هم ساختار مفهومی و هم ساختار واقعی حاصل از سنتز را پوشش می‌دهد.

۴.۸ نتیجه کامپایل

```

Analyzing Slow 1200mV 85C Model
332148 Timing requirements not met
332146 Worst-case setup slack is -6.885
332146 Worst-case hold slack is 0.344
332148 No Recovery paths to report
332140 No Removal paths to report
332146 Worst-case minimum pulse width slack is -3.000
Analyzing Slow 1200mV 6C Model
334003 Started post-fitting delay annotation
334004 Delay annotation completed successfully
332123 Deriving Clock Uncertainty. Please refer to report_sdc in the Timing Analyzer to see clock uncertainties.
332148 Timing requirements not met
332146 Worst-case setup slack is -6.081
332146 Worst-case hold slack is 0.305
332140 No Recovery paths to report
332140 No Removal paths to report
332146 Worst-case minimum pulse width slack is -3.000
Analyzing Fast 1200mV 6C Model
332123 Deriving Clock Uncertainty. Please refer to report_sdc in the Timing Analyzer to see clock uncertainties.
332148 Timing requirements not met
332146 Worst-case setup slack is -3.685
332146 Worst-case hold slack is 0.186
332140 No Recovery paths to report
332140 No Removal paths to report
332146 Worst-case minimum pulse width slack is -3.000
332102 Design is not fully constrained for setup requirements
332102 Design is not fully constrained for hold requirements
Quartus Prime Timing Analyzer was successful. 0 errors, 5 warnings
Running Quartus Prime EDA Netlist Writer
Command: quartus eda -read settings files=off -write settings files=off stack processor -c stack processor top
18236 Number of processors has not been specified which may cause overloading on shared machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your OSF to an appropriate value for best performance.
204019 Generated file stack_processor_top.vo in folder "/home/mahdi/Documents/DSOL/experiment 10/quartus/simulation/questa/" for EDA simulation tool
Quartus Prime EDA Netlist Writer was successful. 0 errors, 1 warning
293000 Quartus Prime Full Compilation was successful. 0 errors, 17 warnings

```

شکل ۶: خروجی Quartus و موفقیت کامپایل کامل پروژه

همان‌طور که در شکل ۶ دیده می‌شود، کامپایل کامل با صفر خطا پایان یافته است. هشدارهای زمان‌بندی موجود در تصویر به نبود فایل قیود زمانی و مشخص‌نشدن قید واقعی ساعت مربوط‌اند و مانع انجام تحلیل و سنتز، جانمایی و مسیریابی و تولید خروجی کامپایل نشده‌اند. بنابراین از دید Quartus طراحی قابل سنتز است.

۹ طراحی و روش کار فایل‌های آزمون

در این پروژه دو فایل آزمون^{۱۵} مستقل وجود دارد. کد این فایل‌ها در گزارش درج نشده است، اما هر دو همراه پروژه تحویل داده می‌شوند و نحوه آزمون آن‌ها در ادامه به‌صورت کامل توضیح داده شده است.

۱.۹ آزمون مستقل دستورهای جدید در `stack_cpu_tb.v`

این فایل مستقیماً `stack_cpu.v` را نمونه‌سازی می‌کند و یک مدل ساده حافظه در کنار آن قرار می‌دهد. برای هر آزمون، یک برنامه بسیار کوتاه در حافظه مدل نوشته می‌شود، پردازنده بازنشانی می‌گردد و پس از پایان دستور مورد نظر، محتوای پشته و در صورت نیاز پرچم‌ها بررسی می‌شوند.

۱.۱.۹ آزمون SWAP

دو مقدار متفاوت 12 و A5 به ترتیب روی پشته قرار می‌گیرند. پیش از SWAP، مقدار A5 به عنوان A در بالای پشته و مقدار 12 به عنوان B در جایگاه دوم قرار دارد. پس از اجرای دستور انتظار می‌رود عنصر بالا 12 و عنصر دوم A5 باشد و عمق پشته همچنان دو باقی بماند. این آزمون الزام مستقل دستور SWAP را پوشش می‌دهد؛ هرچند استفاده از این دستور در برنامه نهایی برای تابع ضروری نیست.

۲.۱.۹ آزمون‌های SHL

چهار مقدار الزام‌شده در صورت آزمایش بررسی شده‌اند:

$$0 \rightarrow 0, \quad 1 \rightarrow 2, \quad 63 \rightarrow 126, \quad 127 \rightarrow 254.$$

در هر چهار حالت، یک مقدار روی پشته قرار گرفته، SHL اجرا و مقدار جدید بالای پشته با نتیجه مورد انتظار مقایسه می‌شود. عمق پشته باید یک باقی بماند.

۳.۱.۹ آزمون‌های CMP

سه حالت مورد نیاز با تعریف $T = B - A$ بررسی شده‌اند:

• $B < A$: با $B = 5$ و $A = 9$ ، داریم $T = -4$ ، پس $Z=0$ و $S=1$.

• $B = A$: با $B = 7$ و $A = 7$ ، داریم $T = 0$ ، پس $Z=1$ و $S=0$.

• $B > A$: با $B = 9$ و $A = 5$ ، داریم $T = 4$ ، پس $Z=0$ و $S=0$.

¹⁵Testbench

علاوه بر پرچم‌ها، آزمون بررسی می‌کند که هر دو مقدار روی پشته بدون تغییر باقی مانده باشند.

```
mahdi@mahdi-ASUS-Zenbook-14:~/Documents/DSDL/experiment 10/quartus$ vvp stack_cpu_tb.out

--- SWAP test ---
VCD info: dumpfile stack_cpu_tb.vcd opened for output.
PASS: SWAP

--- SHL tests ---
PASS: SHL 0 -> 0
PASS: SHL 1 -> 2
PASS: SHL 63 -> 126
PASS: SHL 127 -> 254

--- CMP tests ---
PASS: CMP B=5 A=9 | Z=0 S=1
PASS: CMP B=7 A=7 | Z=1 S=0
PASS: CMP B=9 A=5 | Z=0 S=0

=====
CPU TEST SUMMARY
Passed: 8
Failed: 0
=====
ALL CPU TESTS PASSED
stack_cpu_tb.v:353: $finish called at 896000 (1ps)
```

شکل ۷: خروجی آزمون مستقل SWAP، SHL و CMP

شکل ۷ نشان می‌دهد هر هشت آزمون مستقل موفق بوده‌اند:

Passed: 8 Failed: 0

۲.۹ آزمون برنامه کامل در `stack_processor_top_tb.v`

فایل آزمون دوام کل سامانه، شامل پردازنده، حافظه، برنامه زبان ماشین، منطق ورودی و خروجی و دو رمزگشای نمایشگر را بررسی می‌کند. برای هر ورودی، ابتدا مقدار مرجع تابع در خود فایل آزمون محاسبه می‌شود. سپس X روی ورودی قرار گرفته، بازنشانی اعمال می‌شود و فایل آزمون تا یک شدن `done_led` منتظر می‌ماند. پس از آن موارد زیر با مقدار مورد انتظار مقایسه می‌شوند:

• `error_led`:

• رقم پرارزش و کم‌ارزش مبنای شانزده؛

• الگوی هفت‌بیتی هر دو نمایشگر هفت‌قسمتی.

استفاده از عملگر ضرب در محاسبه مقدار مرجع فایل آزمون، بخشی از سخت‌افزار قابل سنتز نیست و فقط برای مقایسه خودکار نتیجه پردازنده استفاده می‌شود. در برنامه زبان ماشین و کدهای اصلی قابل سنتز، محاسبه $2X$ با `SHL` و $3X$ با $X + 2X$ انجام شده است.

خروجی نمایشگرها فعال‌پایین است. الگوهای لازم برای ارقامی که در آزمون‌های الزامی ظاهر می‌شوند در جدول ۶ آمده‌اند.

رقم	الگوی هفت‌بیتی	نمایش شانزدهی الگو
۰	1000000	0x40
۱	1111001	0x79
۲	0100100	0x24
۴	0011001	0x19
۵	0010010	0x12
۶	0000010	0x02
۷	1111000	0x78
۸	0000000	0x00
E	0000110	0x06

جدول ۶: الگوهای نمایشگر هفت‌قسمتی مورد استفاده در آزمون‌های الزامی

۱۰ توضیح کامل آزمون‌های الزامی برنامه نهایی

۱.۱۰ ورودی $X = 0$

این مقدار کوچک‌ترین ورودی معتبر و در ابتدای بازهٔ اول است. با مقایسهٔ 0 و 32، پرچم علامت برای 32 - 0 یک می‌شود و برنامه به مسیر اول می‌رود:

$$Y = 3(0) + 5 = 5 = 05_{16}.$$

پس رقم پرارزش 0 و رقم کم‌ارزش 5 است. خروجی واقعی دو رمزگشا به‌ترتیب `seven_segment_high=1000000` (0x40) و `seven_segment_low=0010010` (0x12) خواهد بود. `error_led=0` و پس از تکمیل نوشتن خروجی‌ها `done_led=1` می‌شود.

۲.۱۰ ورودی $X = 31$

این مقدار آخرین عضو بازهٔ اول است:

$$Y = 3(31) + 5 = 98 = 62_{16}.$$

بنابراین رقم پرارزش 6 و رقم کم‌ارزش 2 است و الگوهای هفت‌بیتی به‌ترتیب (0x02) `0000010` و `0100100` (0x24) هستند. خروجی معتبر است، پس خطا صفر و پایان در انتهای محاسبه یک می‌شود.

۳.۱۰ ورودی $X = 32$

این مقدار نخستین عضو بازهٔ دوم است. مقایسهٔ اول نشان می‌دهد $X < 32$ برقرار نیست و مقایسهٔ دوم، $X < 64$ را تأیید می‌کند:

$$Y = 2(32) + 20 = 84 = 54_{16}.$$

رقم پرارزش 5 با الگوی 0010010 یا 0x12 و رقم کم‌ارزش 4 با الگوی 0011001 یا 0x19 نمایش داده می‌شود. خطا صفر است.

$$X = 53 \quad \text{۴.۱۰ ورودی}$$

این مقدار بزرگ‌ترین خروجی معتبر بازه دوم را ایجاد می‌کند:

$$Y = 2(53) + 20 = 126 = 7E_{16}.$$

بیت علامت نتیجه هشت‌بیتی صفر است؛ بنابراین مسیر خطا انتخاب نمی‌شود. رقم پرارزش 7 و رقم کم‌ارزش E هستند. الگوی رقم پرارزش (0x78) 1111000 و الگوی رقم کم‌ارزش (0x06) 0000110 است. error_led=0 و done_led=1 در پایان برقرار می‌شوند.

$$X = 54 \quad \text{۵.۱۰ ورودی}$$

این ورودی نخستین مقدار بازه دوم است که خروجی ریاضی آن از ۱۲۷ بزرگ‌تر می‌شود:

$$Y = 2(54) + 20 = 128.$$

عدد ۱۲۸ در نتیجه هشت‌بیتی دارای بیت شماره هفت برابر یک است؛ بنابراین بعد از ADD، پرچم S یک شده و JS مسیر خطا را انتخاب می‌کند. در مسیر خطا مقدار ریاضی ۱۲۸ روی نمایشگر قرار نمی‌گیرد؛ بلکه هر دو رقم صفر می‌شوند. بنابراین HEX=00 و الگوی هر دو نمایشگر (0x40) 1000000 است. هم‌زمان error_led=1 و در پایان done_led=1 می‌شود.

$$X = 63 \quad \text{۶.۱۰ ورودی}$$

این مقدار آخرین عضو بازه دوم است:

$$Y = 2(63) + 20 = 146 > 127.$$

در نتیجه همانند آزمون قبل، مسیر خطا اجرا می‌شود. خروجی دو نمایشگر 00، هر دو الگوی هفت‌بیتی 0x40، خطا برابر یک و پایان برابر یک خواهد بود.

$$X = 64 \quad \text{۷.۱۰ ورودی}$$

این مقدار نخستین عضو بازه سوم است:

$$Y = 64 - 40 = 24 = 18_{16}.$$

رقم پرارزش 1 با الگوی

$$1111001 = 0x79$$

و رقم کم ارزش 8 با الگوی

$$0000000 = 0x00$$

نمایش داده می شود. خروجی معتبر است؛ بنابراین خطا صفر و پایان یک است.

$$X = 127 \text{ ورودی } 8.10$$

این مقدار بزرگ ترین ورودی مثبت هشت بیتی علامت دار است و در بازه سوّم قرار دارد:

$$Y = 127 - 40 = 87 = 57_{16}.$$

رقم پر ارزش 5 با الگوی 0x12 و رقم کم ارزش 7 با الگوی 0x78 نمایش داده می شود. done_led=1 و error_led=0 است.

$$X = 128 \text{ ورودی } 9.10$$

الگوی هشت بیتی 10000000 در نمایش متمم دو برابر 128- است. از آنجا که بیت شماره هفت ورودی یک است، برنامه قبل از ورود به هر بازه تابع، ورودی را نامعتبر تشخیص داده و به مسیر خطا می رود. خروجی دو نمایشگر 00 است و الگوی هر دو نمایشگر 0x40 خواهد بود. همچنین done_led=1 و error_led=1 می شوند.

$$X = 255 \text{ ورودی } 10.10$$

الگوی 11111111 در نمایش متمم دو برابر 1- است؛ بنابراین همان بررسی اولیه علامت، ورودی را نامعتبر تشخیص می دهد. هیچ یک از سه رابطه تابع اجرا نمی شوند. نمایشگرها 00، خطا یک و پایان یک خواهند بود.

X	نتیجه ریاضی	HEX	خطا	الگوی پر ارزش	الگوی کم ارزش	خطا	پایان
0	5	05	خیر	0x40	0x12	0	1
31	9A	62	خیر	0x02	0x24	0	1
32	84	54	خیر	0x12	0x19	0	1
53	126	7E	خیر	0x78	0x06	0	1
54	128	00	بله	0x40	0x40	1	1
63	146	00	بله	0x40	0x40	1	1
64	24	18	خیر	0x79	0x00	0	1
127	87	57	خیر	0x12	0x78	0	1
128	نامعتبر	00	بله	0x40	0x40	1	1
255	نامعتبر	00	بله	0x40	0x40	1	1

جدول ۷: خلاصه خروجی ده آزمون الزامی

```

mahdi@mahdi-ASUS-Zenbook-14:~/Documents/DSDL/experiment 10/quartus$ vvp stack_processor_top_tb.out

=====
MANDATORY PROGRAM TESTS
=====
VCD info: dumpfile stack_processor_top_tb.vcd opened for output.
PASS: X=0 | HEX=05 | error=0
PASS: X=31 | HEX=62 | error=0
PASS: X=32 | HEX=54 | error=0
PASS: X=53 | HEX=7e | error=0
PASS: X=54 | HEX=00 | error=1
PASS: X=63 | HEX=00 | error=1
PASS: X=64 | HEX=18 | error=0
PASS: X=127 | HEX=57 | error=0
PASS: X=128 | HEX=00 | error=1
PASS: X=255 | HEX=00 | error=1

=====
EXHAUSTIVE TEST: X = 0..255
=====

=====
SYSTEM TEST SUMMARY
Passed: 266
Failed: 0
=====
ALL SYSTEM TESTS PASSED
stack_processor_top_tb.v:332: $finish called at 686696000 (1ps)

```

شکل ۸: خروجی فایل آزمون برنامه کامل؛ ده آزمون الزامی و پیمایش جامع ۲۵۶ ورودی

در شکل ۸ تمام ده ورودی الزام شده با نتیجه مورد انتظار موفق شده‌اند. پس از آن نیز تمامی ۲۵۶ الگوی ممکن ورودی هشت‌بیتی از صفر تا ۲۵۵ بررسی شده‌اند. در مجموع ۲۶۶ اجرای مستقل انجام شده و نتیجه برابر است با

Passed: 266 Failed: 0

این آزمون جامع نشان می‌دهد صحت سامانه فقط به چند ورودی مرزی محدود نیست.

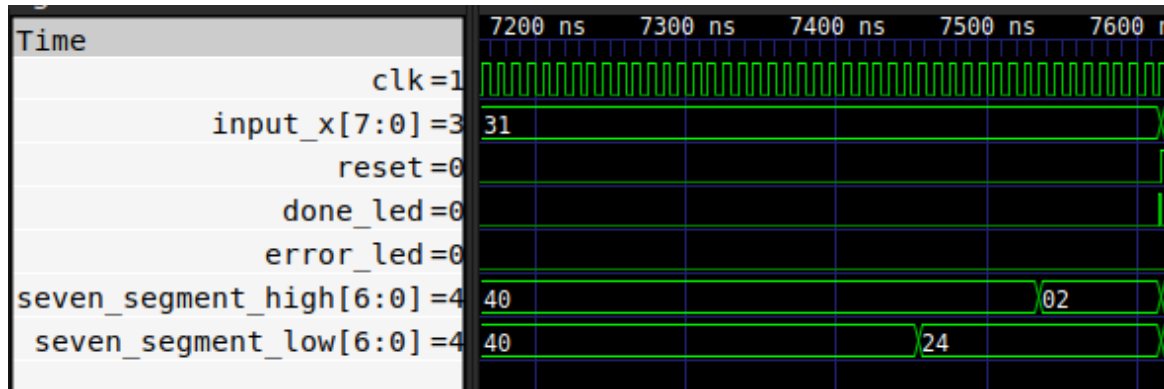
۱۱ شکل موج‌های منتخب و تحلیل آن‌ها

از آنجا که ده آزمون الزامی و سپس ۲۵۶ آزمون جامع اجرا می‌شوند، قرار دادن شکل موج مستقل برای تمام اجراها باعث شلوغی بسیار زیاد گزارش می‌شود. بنابراین در این بخش فقط چند شکل موج^{۱۶} منتخب از مسیرهای مهم آورده شده‌اند. صحت تمام ده آزمون در بخش قبل و در خروجی متنی شکل ۸ ثبت شده است.

در شکل موج‌ها، سیگنال‌های seven_segment_high و seven_segment_low خود عدد نیستند؛ بلکه الگوی هفت‌بیتی فعال‌پایین برای روشن و خاموش کردن قسمت‌های نمایشگر هستند. به همین دلیل برای نمونه نمایش رقم 6 به صورت 0x02 و نمایش رقم 2 به صورت 0x24 دیده می‌شود.

¹⁶Waveform

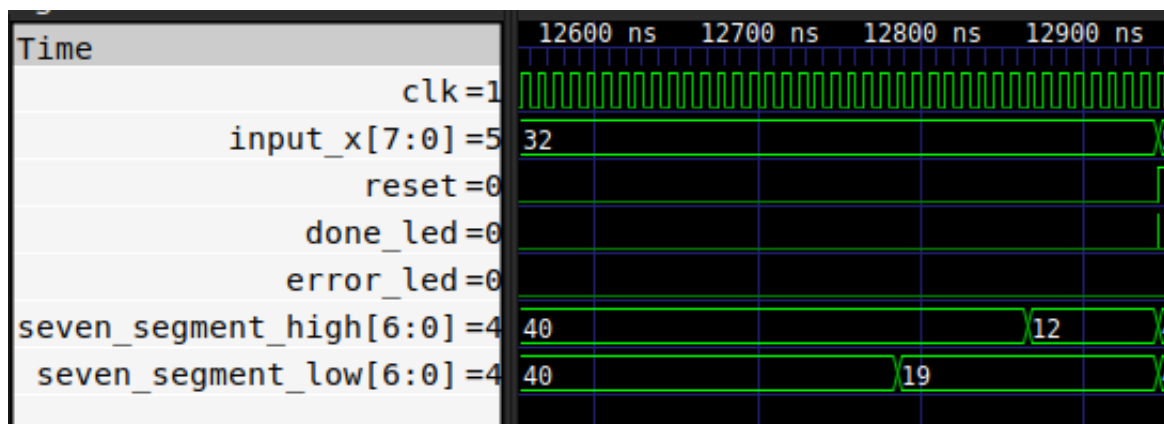
۱.۱۱ شکل موج $X = 31$



شکل ۹: شکل موج اجرای برنامه برای $X = 31$

برای $X = 31$ ، نتیجه تابع $62_{16} = 98$ است. در شکل ۹ ابتدا رقم کم ارزش به $0x24$ ، یعنی الگوی رقم ۲، و سپس رقم پر ارزش به $0x02$ ، یعنی الگوی رقم ۲، تغییر می‌کند. **error_led** صفر باقی می‌ماند و پس از تثبیت خروجی‌ها **done_led** یک می‌شود.

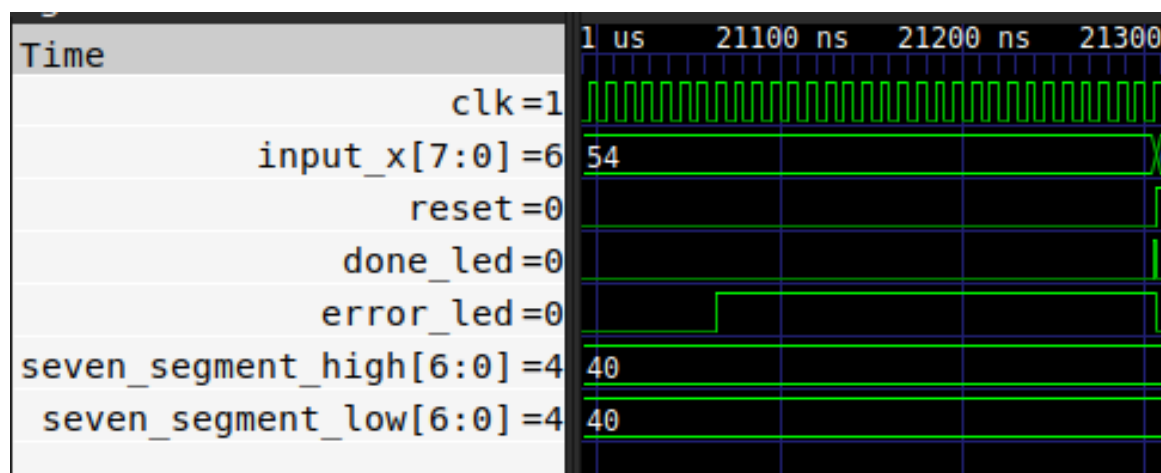
۲.۱۱ شکل موج $X = 32$



شکل ۱۰: شکل موج اجرای برنامه برای $X = 32$

این ورودی نخستین عضو بازه دوم است و خروجی $54_{16} = 84$ تولید می‌شود. الگوی رقم ۵ برابر $0x12$ و الگوی رقم ۴ برابر $0x19$ است. در شکل ۱۰ هر دو مقدار پیش از فعال شدن **done_led** روی خروجی قرار گرفته‌اند و خطا صفر است.

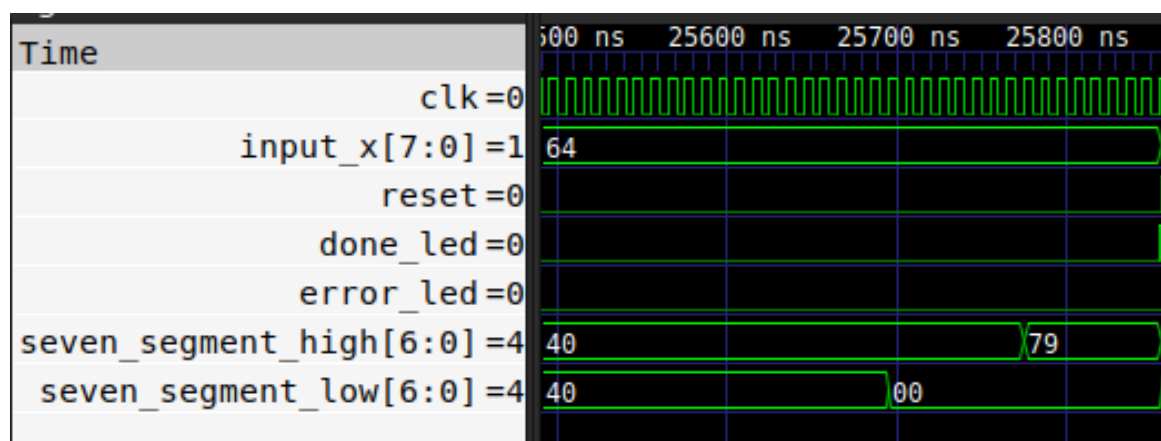
۳.۱۱ شکل موج $X = 54$



شکل ۱۱: شکل موج مسیر خطا برای $X = 54$

برای $X = 54$ ، خروجی ریاضی ۱۲۸ است و بنابراین سرریز^{۱۷} نسبت به بازه مجاز مثبت رخ می‌دهد. شکل ۱۱ نشان می‌دهد که $error_led$ فعال می‌شود و هر دو نمایشگر روی الگوی 0x40، یعنی رقم صفر، باقی می‌مانند. در پایان $done_led$ نیز یک می‌شود.

۴.۱۱ شکل موج $X = 64$



شکل ۱۲: شکل موج اجرای برنامه برای $X = 64$

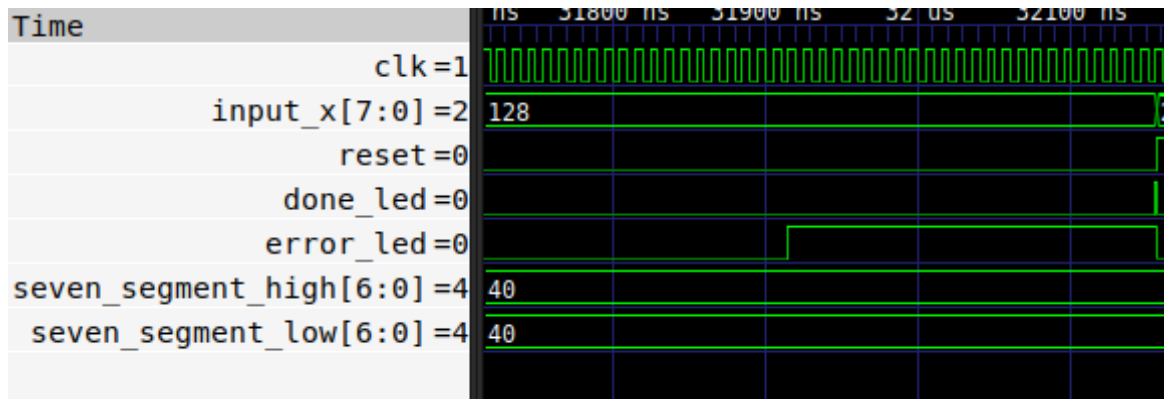
برای $X = 64$ ، مسیر سوّم تابع اجرا می‌شود:

$$Y = 64 - 40 = 24 = 18_{16}.$$

در نتیجه الگوی رقم پرارزش 1 برابر 0x79 و الگوی رقم کم‌ارزش 8 برابر 0x00 است. هر دو مقدار در شکل ۱۲ دیده می‌شوند و خطا صفر باقی می‌ماند.

¹⁷Overflow

۵.۱۱ شکل موج $X = 128$



شکل ۱۳: شکل موج تشخیص ورودی منفی برای $X = 128$

الگوی 128_{10} ، یعنی $0x80$ ، در نمایش علامت‌دار متمم دو برابر $128 -$ است. بنابراین برنامه مستقیماً مسیر خطا را انتخاب می‌کند. در شکل ۱۳ $error_led$ یک می‌شود، هر دو نمایشگر الگوی صفر $0x40$ را حفظ می‌کنند و در پایان $done_led$ فعال می‌شود.

۱۲ انطباق با الزامات صورت آزمایش

برای اطمینان از پوشش کامل خواسته‌های آزمایش، جدول ۸ نحوه برآورده شدن هر الزام را خلاصه می‌کند.

ردیف	الزام	نحوه برآورده شدن
۱	توسعه مسیر داده و واحد کنترل برای SWAP/SHL/CMP	بخش‌های ۳ و ۴ و جدول ریز عملیات
۲	قابل سنتز بودن تمام بخش‌ها	کامپایل کامل Quartus با صفر خطا، شکل ۶
۳	استفاده از CMP برای تشخیص بازه‌ها	مقایسه X با صفر، ۳۲ و ۶۴ در برنامه زبان ماشین
۴	استفاده از SHL برای $2X$	هر دو مسیر $3X + 5$ و $2X + 20$ از SHL استفاده می‌کنند
۵	آزمون مستقل SWAP	$stack_cpu_tb.v$ و شکل ۷
۶	عدم استفاده از ضرب مستقیم در پیاده‌سازی تابع	$2X$ با SHL و $3X = X + 2X$: ضرب مدل مرجع فابل آزمون جزئی از سخت‌افزار نیست
۷	ارتباط ورودی و خروجی فقط با نگاشت حافظه‌ای	نشانی‌های F8-FC مطابق جدول ۳
۸	گزارش حداکثر عمق پشته	حداکثر عمق برابر ۲ و استدلال آن در بخش مربوطه
۹	جدول کامل حافظه برنامه	جدول ۵
۱۰	نمودار روند برنامه زبان ماشین	شکل ۳

جدول ۸: انطباق طراحی با الزامات پیاده‌سازی

آزمون‌های مستقل SHL برای مقادیر ۰، ۱، ۶۳ و ۱۲۷ و سه حالت CMP یعنی $B < A$ ، $B = A$ و $B > A$ نیز همگی در بخش آزمون‌های دستورها پوشش داده شده‌اند. ده ورودی الزامی برنامه نهایی نیز به صورت جداگانه تحلیل و نتیجه

واقعی نمایشگرها برای هرکدام ارائه شده است.

۱۳ فایل‌های تحویلی

فایل‌های اصلی پروژه عبارت‌اند از:

- stack_cpu.v
- memory_io.v
- processor_system.v
- seven_segment_decoder.v
- stack_processor_top.v
- program.hex
- stack_cpu_tb.v
- stack_processor_top_tb.v

فایل‌های آزمون در گزارش به صورت کد کامل درج نشده‌اند، اما همراه نسخه نهایی پروژه تحویل داده می‌شوند. فایل‌های Verilog اصلی نیز با توضیحات کوتاه و انگلیسی درون کد نگهداری شده‌اند.

۱۴ جمع‌بندی

در این آزمایش پردازنده پشته‌ای پایه با سه دستور SWAP، SHL و CMP توسعه یافت و مسیر داده و واحد کنترل برای اجرای آن‌ها اصلاح شد. سپس برنامه زبان ماشین تابع چندبخشی خواسته شده را بدون استفاده از عملگر ضرب در سخت‌افزار اجرا کرد. تشخیص بازه‌ها با CMP و پرش‌های شرطی، محاسبه $2X$ با SHL و استخراج دو رقم مبنای شانزده با کم کردن تکراری ۱۶ انجام شد.

حد اکثر عمق پشته در تمام مسیرها دو است. آزمون مستقل دستورهای جدید با هشت بررسی و آزمون کامل سامانه با ده ورودی الزامی و سپس تمام ۲۵۶ ورودی ممکن اجرا شد. نتیجه آزمون‌ها به ترتیب 8/8 و 266/266 بود. همچنین پروژه در Quartus با صفر خطای کامپایل کامل شد و نمای سطح انتقال ثبات ساختار سنتز شده را تأیید کرد. مجموعه این نتایج صحت عملکرد پردازنده، برنامه زبان ماشین و رابط‌های ورودی و خروجی را در سطح شبیه‌سازی و سنتز نشان می‌دهد.